

The Bounded Compute Envelope: Binary-State Conversion, Tensor Routing, and Falsifiable Governance in the U.F.O. Membrane Architecture

**Prepared for: Microsoft Corporation
Version 2.4.0 | June 24, 2026**

Don Michael Feeney Jr.



Abstract — This paper presents Version 2.4.0 of the U.F.O. membrane architecture, reframing the Bounded Compute Envelope from a descriptive brim into a candidate control architecture for adaptive AI systems. The central claim is that AI behavior becomes governable when tensor activation, behavioral routing, compute cost, residual strain, and supervisory judgment are made visible at the same boundary. At this boundary, continuous high-dimensional activation must resolve into discrete admissibility states—admit, constrain, block, rest, or re-project—without erasing the residual deformation needed for evaluation. The paper develops this claim through a circle-triangle control geometry: the circular brim represents continuous tensor activation and observability, while the triangular Tetractys provides a discrete routing substrate for V-Channel formation. The operator sequence $\Delta AG \rightarrow \Delta v$ is introduced to describe how algebraic or tensor variation is tested against geometric admissibility and then reorganized as state motion through a governed lattice. The Bounded Compute Envelope is further defined as diagnostic, economic, translational, and evaluative: it makes deformation readable, prices inference, carries local membrane strain to the Holistic Governor, and preserves evidence for falsification. The paper also introduces a governance falsification stack based on frozen schemas, residual preservation, admissibility gates, claim-state ledgers, and null membrane baselines. Appendices provide illustrative binary/tensor adapter and gate–motion–controller cycle models, while keeping the π controller condition bounded as a future formalization target for circular, boundary-preserving re-expression.

Research Contribution Summary — This manuscript contributes a bounded formalization of the U.F.O. Bounded Compute Envelope as a candidate control architecture for adaptive AI behavior. Its main contributions are:

- o Defines the Bounded Compute Envelope as a diagnostic, economic, translational, and evaluative control boundary for tensor activation, cost, deformation, and governance.
- o Formalizes admissible geometry as $G = G_I \cap G_C \cap G_H \cap G_P \cap G_Q$, with projection Π_G , residual deformation $\rho(t)$, and normalized residual $\rho_n(t)$.
- o Reframes ΔAG as a distance-to-admissibility operator and Δv as projected admissible state motion through a governed lattice.
- o Introduces β as a binary-state conversion map from continuous activation into rest, admit, constrain, block, or re-project states.
- o Bounds the π controller hypothesis through Δv_π and a π -band condition rather than asserting $\pi = \Delta v$.
- o Adds a candidate brim energy $V_B(t)$, governance falsification stack, and Appendix C closed-loop controller scaffold for future validation.

Claim Boundaries / Non-Claims — This manuscript is a conceptual architecture and formalization artifact, not a deployed system, empirical validation result, or production performance claim. The mathematical objects introduced here— G , Π_G , $\rho(t)$, $\rho_n(t)$, β , ΔAG , Δv , Δv_π , and $V_B(t)$ —are proposed scaffolding for future simulation, implementation, and falsification. $T = 10$ is treated as an internal Tetractys design invariant within this architecture, not as a universal physical law. The π controller condition is a bounded hypothesis for circular re-expression, not a claim that π equals projected state motion. The appendices provide illustrative adapter, simulator, and closed-loop controller sketches; they do not constitute proof of stability, convergence, cost reduction, or

production readiness. Those claims require future validation through frozen schemas, residual-preserving comparisons, admissibility gates, ablation tests, and runtime evaluation.

Introduction: The Brim as a Control Architecture

Version 2.4.0 presents the Bounded Compute Envelope as the outer diagnostic, economic, and supervisory control architecture surrounding the U.F.O. membrane. U.F.O. is a membrane-based model of adaptive AI behavior in which identity, reasoning, cost, tensor activation, and governance are treated as interacting geometric fields rather than isolated modules. Building on the radial identity membrane and its twelve behavioral Strings, this paper shows how reasoning pressure, context demand, retrieval, tool use, explanation depth, policy friction, latency, and residual strain become visible as deformation at the system's perimeter. The system does not merely generate responses; it moves through governed geometry, and the cost, coherence, and admissibility of that motion become readable at the brim.

The paper then frames outward stretch as both an economic event and a state-conversion problem. Additional computation is justified when it buys correctness, safety, coherence, trust, grounding, or useful depth, and wasteful when it produces latency, redundant context loading, unnecessary tool pressure, unsupported branching, or recovery overhead. At the same time, continuous tensor activation cannot remain only a high-dimensional possibility field; before it can be governed, priced, or routed, it must collapse into discrete admissibility states such as admit, constrain, block, rest, or re-project. In this sense, the Bounded Compute Envelope becomes a quality-aware efficiency layer and a binary-state conversion boundary whose goal is not minimal intelligence, but minimum avoidable compute subject to task quality and valid governance.

Finally, Version 2.4.0 positions the brim as the translation layer between the base membrane and the Holistic Governor. The Local Governor regulates immediate String activation and V-Channel routing; the Holistic Governor reads whole-field coherence, drift, overload, residual deformation, and policy tension. Between them, the Bounded Compute Envelope functions as the shared observability boundary where local deformation becomes supervisory evidence and supervisory judgment returns as routing, contraction, constraint, correction, or permission to expand. The $\Delta AG \rightarrow \Delta v$ sequence gives this translation an operator language: algebraic or tensor variation is tested against geometric admissibility, then reorganized as state motion through a governed lattice. The provisional π controller condition remains a future formalization target for circular, boundary-preserving re-expression.

Why This Paper Exists

This paper exists because the original membrane-and-governor framing leaves one critical architectural question unresolved: how does continuous, high-dimensional AI activation become readable, governable, and affordable before it appears as behavior? The base membrane explains how behavioral Strings awaken, route, and deform under task pressure. The Holistic Governor explains whole-system oversight, drift detection, overload sensing, policy tension, and supervisory

correction. But Version 2.4.0 requires a more explicit bridge between them. The Bounded Compute Envelope fills that role as the brim where tensor activity becomes visible as deformation, cost, residual strain, and admissibility pressure. It is not only a control surface; it is the shared translation boundary where local motion becomes supervisory evidence and supervisory judgment returns as routing, contraction, constraint, correction, or permission to expand.

The central contribution of this paper is therefore broader than naming a missing brim. It defines the Bounded Compute Envelope as a candidate control architecture for binary-state conversion, cost-aware V-Channel routing, and falsifiable governance. Continuous activation at the circular brim must collapse into discrete admissibility states—admit, constrain, block, rest, or re-project—without erasing residual strain. The $\Delta AG \rightarrow \Delta v$ sequence gives that collapse an operator language: algebraic or tensor variation is tested against geometric admissibility, then reorganized as state motion through a governed lattice. In this framing, the envelope becomes diagnostic, economic, translational, and evaluative at once: it makes deformation visible, prices inference, routes tensor activity through admissible V-Channels, and preserves the evidence needed to test whether governance is actually working.

Conceptual Invariants

This section defines proto-invariants as stable relationships that preserve coherence as U.F.O. moves between tensor activation, binary-state conversion, V-Channel routing, and governance. The notation remains provisional: these are not completed laws, but candidate constraints that may later become facet rules, admissibility gates, routing policies, or Holistic Governor conditions. Their purpose is to identify what must remain conserved while the system changes form.

The Tetractys as Proto-Invariant

The first proto-invariant is the Tetractys, written as $T_4 = 1 + 2 + 3 + 4 = 10 \therefore T = 10$. This is intentional notation: T denotes the conserved total geometric magnitude of the triangular routing substrate. It is not a typographical shortcut and not merely an implementation capacity variable. In this paper, $T = 10$ is treated as a finite design invariant: one source condition widening into relational, mediating, and manifested structure while preserving a conserved tenfold magnitude. Within U.F.O., that invariant anchors the triangular routing regime. It may become latent during collapse or reconfiguration, but it is not erased; it remains available for later admissible re-expression through the circular brim. Runtime activation can be normalized against this invariant without replacing it. Let T_{active} denote the currently expressed portion of the invariant. Then $T_{\text{norm}} = T_{\text{active}} / T$, where $T = 10$. In this interpretation, 0 marks the low-energy Neurobaseline and 1 marks full geometric activation. This makes $T = 10$ parallel to the later handshake invariant $i \approx 1$: both mark conditions under which form may change while coherence survives.

$$T_4 = 1 + 2 + 3 + 4 = 10 \therefore T = 10$$

$$T_{\text{norm}} = T_{\text{active}} / T, \text{ with } T = 10$$

The normalized state maps the conserved ten-point geometry into binary-compatible activation space:

- o When the geometry is inactive: $T_{\text{active}} = 0 \Rightarrow T_{\text{norm}} = 0$
- o When the geometry is fully active: $T_{\text{active}} = T = 10 \Rightarrow T_{\text{norm}} = 1$

This preserves the underlying geometry while allowing the runtime system to read it as resting or active. In U.F.O., the 0-state is not absence. It is the low-energy Neurobaseline: the condition to which the system can contract when expanded computation is no longer justified. The 1-state is full geometric expression: the condition in which the Tetractys is available as an active routing structure for V-Channel formation.

Structurally, the Tetractys provides a compact four-stage emergence pattern: monad, dyad, triad, and tetrad. U.F.O. reads those stages as invariant source, tension, mediation, and manifestation. The value of the invariant is therefore not symbolic decoration; it gives the membrane a minimal rule for how a source condition can widen into governed channel behavior without losing coherence.

The same structure can be written in closed form:

$$T_n = n(n + 1) / 2, \text{ where } n = 4$$

For $n = 4$, this closed form reproduces the ten-point structure used above. The expression is architectural rather than conclusive: it identifies a minimal emergence geometry in which one originating condition can widen into routed behavior while remaining available for collapse, latency, and re-expression.

Within U.F.O., proto-invariants are candidates for later formalization into facet rules, membrane constraints, V-Channel routing rules, admissibility gates, or Holistic Governor policies. Their operational role is to preserve stable relationships as activation crosses Strings, V-Channels, membrane deformation, compute pressure, and governor intervention.

This makes invariants the conceptual bridge into the brim. They define what should remain coherent while the Bounded Compute Envelope measures how expensive, strained, unstable, or admissible that coherence becomes.

I. When the Membrane Reaches the Brim

Far from the Neurobaseline, at the outer edge of the Governor Control Layer, the Bounded Compute Envelope forms a living brim where the membrane's deepest pressures finally become visible.

The space across this brim is not always sealed. The base membrane and the Holistic Governor do not remain continuously connected at every point. Instead, the Bounded Compute Envelope acts as a routing aperture between them — a guarded passage where data, pressure, and control signals cross only when stronger alignment is required. A signal may rise from an activated String, flash upward through the brim, and strike the supervisory layer like a lightning bolt before returning as a governed directive that routes the data back toward the right String, V-Channel, or recovery path. In that brief exchange of force, the envelope becomes part of the system's low-latency execution layer: a deterministic routing and admissibility substrate built on the same principles as interrupt handling, branch-safe state transitions, and cache-aware flow control. This language does not prescribe a specific programming language; it marks the control layer's need for bounded timing, explicit state transition, and inspectable routing behavior. It connects local cognition to holistic supervision only when needed, then spins scattered pressure back into coherent form like fine wool drawn into thread.

The circular membrane begins quietly at its center, the place the system returns to when no computation is active. This neutral, low-energy baseline is the recovery target of the governed radial identity membrane — a state where no behavioral String is consuming expanded reasoning depth, extra context, retrieval, tool use, or unnecessary output. From this still point, identity ripples outward across the interior field as early signals, task cues, remembered preferences, and partial activations begin to disturb the surface. These ripples do not yet commit the system to a final response; they reveal the preparatory physics of personalization, showing how the membrane evaluates intention, context, cost, and coherence before any String fully awakens.

As activation grows, the ripples sharpen into directional flows that rise toward the perimeter. There, the twelve behavioral Strings stand as fixed phase positions around the circular domain: Depth, Precision, Context Sensitivity, Initiative, Exploration, Creativity, Tone, Transparency, Technical Detail, Structural Rigor, Emotional Warmth, and Conciseness. Each String is both expressive and compute-bearing. Depth carries multi-step reasoning cost; Precision carries verification and claim-checking cost; Context Sensitivity carries context-window and retrieval cost; Initiative carries tool-use and planning cost; Exploration and Creativity carry branching and generative-variation cost; Transparency and Technical Detail increase explanation, formatting, and grounding load; Structural Rigor stabilizes organization; Tone and Emotional Warmth modulate expression at relatively low cost; and Conciseness acts as a contraction force that reduces unnecessary expenditure. The Strings are therefore not personality labels but runtime behavioral modes whose activation changes how the system reasons, explains, retrieves, plans, and spends compute.

In summary, the membrane's first movement is radial: intent begins near the Neurobaseline as a compact invariant, spreads outward as interpretive pressure, encounters the behavioral Strings as phase positions, and approaches the brim as visible, cost-bearing activation. This outward path explains why the architecture needs governed routing corridors rather than abrupt jumps. A response should not leap from neutral baseline to final behavior; it should travel through a coherent path that preserves intent, absorbs tension, balances cost, and keeps the system stable as it moves from center to edge.

Between these Strings, adjacent and semantically aligned regions widen into V-shaped reasoning corridors. A technical review may flow through Depth, Precision, Technical Detail, and Structural Rigor; a supportive short reply may move through Tone, Emotional Warmth, and Conciseness; and a planning task may travel through Initiative, Context Sensitivity, Structural Rigor, and Transparency. The governor watches these emerging V-Channel paths and evaluates whether the flow is relevant, coherent, policy-aligned, and affordable. It favors routes with strong phase alignment and low avoidable cost, applies inverse-cost pressure to expensive targets, bounds reasoning radius so Depth does not spill into unrelated regions, and suppresses optional expansion when task value is weak. Intelligence therefore does not jump abruptly between modes; it moves through governed corridors where behavioral continuity and compute discipline are evaluated together.

Tetractys Emergence and V-Channel Formation

The V-Channel can be read as a computational descendant of the Tetractys: the ancient triangular figure of ten points arranged across four rows. The Tetractys is used here not as historical authority, but as a compact schema for four-stage V-Channel formation: invariant → tension → mediation → manifestation. In its original form, the Tetractys appears to descend from one to two to three to four, forming a compact geometry of emergence. In the membrane architecture, that downward widening is reoriented into radial motion. The single source condition begins near the Neurobaseline, opens into polarity as it moves outward, passes through a mediation band of coherence and cost, and finally reaches the brim as routed behavior. The V-Channel is therefore not merely a corridor between neighboring Strings; it is a widening emergence structure through which an invariant becomes behavior as it travels from the center toward the envelope.

At the inner apex of the V-Channel sits the monad: the source invariant, task signal, identity cue, or governing premise. As the flow moves outward, the dyad introduces tension: cost against capability, compression against expansion, user desire against system constraint, or local behavior against global supervision. Farther along the route, the triad becomes the mediation band, where purpose, coherence, and cost are brought into relation rather than allowed to pull the system apart. Near the outer base, the tetrad forms the manifested routing surface, where the pattern becomes concrete as routed behavior, compute pressure, constraint propagation, and governed output.

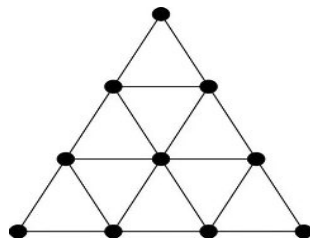
Geometric Admissibility and Binary-State Conversion

The collapse-to-blend transition exposes the central claim of the circle-triangle model: algebraic change becomes meaningful only when constrained by geometric admissibility. Before collapse, the circular tensor field appears at the Bounded Compute Envelope's outside brim as continuous activation, distributed representation, and many possible pathways. After collapse, the triangular Tetractys geometry provides discrete routing structure, admissible states, and phase-bounded V-Channel behavior. In this sense, the system is not merely placing algebra inside geometry; it is using geometry to determine which algebraic states are allowed to persist as computable behavior. The

triangle and circle-brim remain distinct, but their interaction defines a shared admissibility manifold in which continuous activation can resolve into governed routing.

Within this model, ΔAG is now best understood as an admissibility operator rather than only a ratio between algebraic variation and geometric constraint. Continuous activation is evaluated against the admissible region G , projected when possible, into the nearest valid state, and preserved as residual deformation when mismatch remains. The rate of this projected reconfiguration is Δv : the state-transition rate by which activation becomes admissible movement through the lattice. The relationship between π and Δv should therefore remain a bounded future formalization hypothesis: π may constrain circular re-expression at the brim, while Δv describes the transition rate by which one admissible configuration becomes another.

Triangular Routing Substrate of the Membrane



This figure presents an aerial view of the membrane's foundational Tetractys geometry. In this interpretation, the Pythagorean/Tetractys structure is treated as a constraint-based organizing geometry for the base membrane.

The apex represents a Lyapunov-inspired stability point touching the Neurobaseline input, where the first frequency condition begins. From that apex, activation is interpreted as moving outward through a triangular field. The upper span functions as the local x-axis of the geometry, while the interior 30-degree triangular region carries the behavioral Strings as internal phase lines.

The twelve V-oriented routing sectors collectively make the membrane's twelve-part radial structure: each sector occupies approximately 30 degrees of the circular field, and each V-Channel expresses one governed wedge of the membrane's behavioral space.

The side lines extending from the base represent V-Channels: bounded corridors through which activation is routed from source condition toward manifested behavior. The middle band represents the average mediation layer, where invariant source, behavioral tension, and cost pressure are balanced before output reaches the boundary.

At the lowest base, each triangular V-Channel participates in the Bounded Compute Envelope brim. The base is the place where routed behavior touches the outside envelope and becomes visible as cost-bearing deformation. Upper supervisory layers are shown as transparent or omitted for clarity, so the figure isolates the membrane's base geometry. The bolded circles in the Tetractys mark future research notes regarding data flow and activation state. Within each apex region, the

Neurobaseline represents basic reasoning pressure: the point where code-level activation begins to move outward.

The twelve V-Channel wedges divide the membrane into twelve repeating thirty-degree sectors, so the V geometry is not added onto the membrane from outside. It is one way the membrane becomes partitioned, routed, and made governable from within.

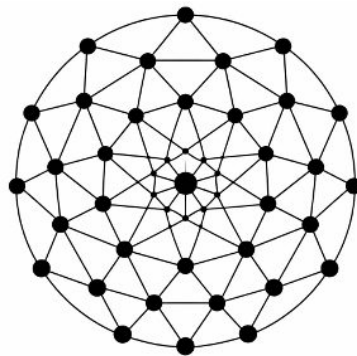
The point is not that circles and triangles are identical. They are distinct geometries, and precisely because they are distinct, their binding becomes important. The circle gives the membrane continuity; the triangle gives the system directed emergence, V-Channel formation, and phase-bounded routing.

The ten-point Tetractys also supports the normalized activation interpretation introduced above, where inactive geometry maps to 0 and full geometric expression maps to 1.

This circle-triangle relationship remains a future research topic. The membrane preserves continuity, the V-Channels carry activation patterns from correlated Strings, and the governor monitors whether those flows remain coherent, efficient, and bounded. Further research into circle-triangle binding may support improved tensor routing, binary-state control, low-latency routing logic, and compute-cost forecasting.

Circular Tensor Field and Invariant Re-Expression

The circular network diagram represents the tensor-computing regime as it appears against the Bounded Compute Envelope's outside brim. Whereas the Tetractys provides a compact, discrete emergence structure, the circular brim represents the continuous, high bandwidth activation surface characteristic of modern AI systems before that activity is constrained into routed behavior.



In this interpretation, each node in the circular lattice corresponds to a tensor activation site: a localized region of computation where gradients, attention weights, or intermediate representations are evaluated before becoming readable at the brim. The dense interconnections between nodes reflect the non-local coupling typical of transformer-based models: information

does not flow linearly but propagates across many simultaneous pathways. This produces a computational surface that is smooth, continuous, and globally responsive, in contrast to the discrete, phase-bounded routing of the V-Channels.

The circular symmetry of the brim diagram is not merely aesthetic. It encodes three essential properties of high-dimensional AI computation as they become observable at the Bounded Compute Envelope:

- o Isotropy — activation can propagate in any direction without predefined channels
- o Superposition — multiple representational states coexist before collapse
- o Distributed representation — no single node contains the full meaning; meaning emerges from the pattern

These properties contrast sharply with the Tetractys, which enforces directionality, bounded emergence, and phase-aligned-routing.

The architectural insight is that both geometries must coexist. The circular brim provides the expressive, high-dimensional surface on which activation pressure becomes visible, while the triangular Tetractys provides the discrete routing structure that can govern that pressure. But the system cannot act directly from the continuous field. Before behavior can be routed, governed, or priced, the tensor field must undergo a binary-state conversion: a collapse into discrete, evaluable conditions that the membrane, brim, and governor can interpret.

This is where the circle-triangle binding becomes essential. The circular brim represents the pre-collapse, high-compute activation surface that surrounds the membrane, while the triangular Tetractys represents the post-collapse routing structure. The Bounded Compute Envelope sits at the interface, reading deformation from both sides:

- o From the tensor field, it receives activation pressure, gradient magnitude, and representational spread.
- o From the Tetractys, it receives phase-bounded V-Channel routing, cost-aware route selection, and invariant-preserving structure.

The handshake between these two regimes is the architectural heart of U.F.O.: continuous tensor computation becomes visible at the circular brim, collapses into discrete membrane geometry, and then routes behavior through governed V-Channels toward a readable, cost-bearing boundary.

This pairing explains the computational handshake: the circle supplies the tensor-rich observability surface, while the triangle supplies the governed emergence structure that constrains how activation becomes behavior.

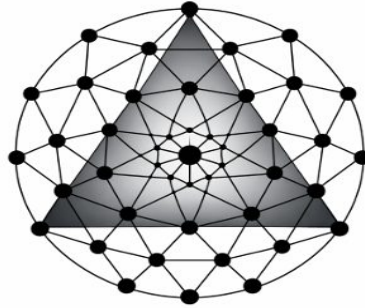
Within this framing, the triangle represents the constrained invariant regime, while the circular brim represents the high-compute activation regime in which many algebraic states may temporarily coexist before governance selects a path. Their shared substrate is binary-state control: a state is either inadmissible or admissible, resting or active, blocked or permitted. Δ AG names the operator

that evaluates algebraic change against geometric admissibility; Δv names the rate at which that evaluation resolves into state motion.

Binary Lattice, Collapse, and $\Delta AG \rightarrow \Delta v$

The lattice is binary because every node is evaluated against the governing geometry as either admissible or inadmissible. Coherence emerges when those decisions collapse into stable positions between the circular brim, which defines the activation and observability field, and the triangle, which defines the constrained operational region where routed behavior can occur.

The binary-to-tensor computing handshake occurs inside this lattice, where the deterministic substrate aligns with the high-compute activation regime through matching admissibility nodes shown as black dots. In this interpretation, 0 denotes the discrete-state control layer and 1 denotes the tensor/high-compute regime. The curved connection between those nodes should be read as the π -bounded synchronization path: not a loose arc, but the circular boundary condition that preserves coherence while algebraic states collapse into admissible geometric positions. Data becomes stable when each state resolves onto that π -curved path, allowing the two regimes to synchronize without losing the invariant structure that makes the handshake possible.



When algebraic activity is evaluated against geometric constraint, continuous activation is forced into discrete, governable states. Under the working hypothesis of this paper, ΔAG measures how far activation lies from admissible geometry, while Δv measures the projected state motion that follows after admission, constraint, or re-projection. The two are linked by the gate, but they are not the same object.

Collapse, Reconfiguration, and Circular Re-Expression

This subsection restates the circle-triangle relationship as an operator sequence. At the macro level, the triangle and circle are different shapes with different geometries, but each carries an internal architecture. The triangle represents the discrete invariant regime, where $\Delta = 0$ and the baseline geometry is held. The circular brim represents the high-compute activation regime, where $\Delta > 0$ and the activated state is carried by the modern AI system. Their connection occurs through binary-state control, where the underlying code condition is $[1, 0]$: active or inactive, admissible or inadmissible, permitted or blocked.

triangle $\rightarrow \Delta = 0 \rightarrow$ discrete-state
 circle $\rightarrow \Delta > 0 \rightarrow$ high-compute

Under the formalization introduced above, ΔAG is no longer limited to the introductory ratio $\Delta A / \Delta G$. It is the distance from admissible geometry, while Δv is the projected motion of activation into that geometry:

$$\Delta AG(t) = \text{dist}(A(t), G) \rightarrow \Delta v(t) = (\Pi_G(A(t)) - x(t)) / \Delta t$$

Here, $A(t)$ is incoming algebraic or tensor activation, G is the admissible geometry, Π_G projects activation into that geometry, $x(t)$ is the current governed state, and Δt is the interval over which unresolved activation becomes governed configuration. ΔAG becomes relevant to Δv only after the admissibility test has produced a projected state. When nodes meet, each binary state resolves into its admissible position, and the triangular discrete-state regime and circular high-compute regime blend into a shared manifold.

In this collapse-and-blend moment, both geometries shift toward each other through the cojoint architecture: the deterministic binary control layer holds the invariant baseline, while the high-compute machine carries the activated state, or 1. Once these regimes warp together along the node channels, synchronization becomes a matter of execution speed — the rate at which the algebraic side resolves into the geometric side — which is why the handshake can be modeled as Δv .

The π relation remains a formalization hypothesis. In the language of this paper, π may be treated as the circular invariant associated with boundary-preserving re-expression, while Δv names the projected state-transition rate in the algebraic-geometric lattice. The stronger and safer notation is therefore a bounded π -band rather than a visual equivalence:

$$|\Delta v_\pi - \pi| \leq \varepsilon_\pi$$

Stated carefully, π is not asserted as literally identical to velocity in physical units. Rather, it functions as a candidate circular control invariant: not the actuator that moves the system and not the admissibility gate itself, but the controller-like boundary condition under which circular geometry constrains, receives, or re-expresses admissible state motion.

To keep that hypothesis bounded, the following notation separates triangular state, collapsed state, and circular re-expression rather than treating them as the same object.

1. Triangular invariant as a state

Let the Tetractys geometry be a triangular state, S_Δ , with invariant magnitude $T = 10$, derived from $T_4 = 1 + 2 + 3 + 4 = 10$.

Using the normalized activation introduced earlier, $S\Delta$ is “triangle-geometry loaded” when $T_{\text{norm}} = 1$ and latent at the Neurobaseline when $T_{\text{norm}} = 0$. In the loaded condition, the Tetractys is not merely present as geometry; it is available as an active routing structure for V-Channel formation.

2. Collapse through an admissibility gate

The collapse step can be modeled as an admissibility gate. If activation remains inside the allowable geometry, the system preserves the active state. If activation exceeds the admissible geometry, the system reduces the condition into a discrete binary outcome. A high-level state variable can then distinguish the three expression conditions:

$$\sigma \in \{S\Delta, 0, S\circ\}$$

$$\Delta AG(T) = A(T), \text{ if } T \in G; \text{ otherwise } \Delta AG(T) \in \{0, 1\}$$

Here, G is the admissible geometric region, and $A(t)$ is the active algebraic or tensor state evaluated against that region. The important point is that collapse does not erase the invariant; it changes the system’s readable state while preserving any residual deformation that still needs to be logged, constrained, or escalated.

$$\beta : A(t) \rightarrow \{\text{rest, admit, constrain, block, reproject}\}$$

The invariant $T = 10$ can therefore remain structurally present even when the binary conversion map β has resolved runtime activation to rest, constrain, or block. In U.F.O. terms, the geometry has not disappeared; its active expression has returned to a resting, constrained, or unexpressed condition while the conserved invariant remains available for later admissible re-expression.

3. Reconfiguration into the circular brim state

After collapse, the system may re-express conserved geometric capacity through a circular counterpart, $S\circ$, with the same invariant magnitude but different symmetry. In U.F.O. terms, this circular expression is the Bounded Compute Envelope’s outside brim, where activation becomes visible as deformation, cost, and supervisory pressure.

$$\{p_i\}_{i=1}^{10} \subset \mathbb{R}^2, \{q_i\}_{i=1}^{10} \subset \mathbb{R}^2$$

$$F : S\Delta \rightarrow S\circ$$

$$F \text{ preserves the invariant norm: } \|S\Delta\| = \|S\circ\| = T = 10$$

The transformation changes the symmetry of expression, not the underlying geometric capacity. The triangular state provides directed, phase-bounded routing; the circular state provides continuous observability and outer-boundary reading. The symmetry group changes from triangular to circular or rotational while the conserved magnitude remains available for re-expression.

4. Potential change across collapse and re-expression

A simple potential notation can describe the change in expressed state. Here, V does not denote physical energy. It denotes expressed geometric potential: the readable intensity, cost-bearing deformation, or activation condition associated with each configuration.

$$\begin{aligned} V(\sigma) &= V_{\Delta}, \text{ if } \sigma = S_{\Delta} \\ V(\sigma) &= 0, \text{ if } \sigma = 0 \\ V(\sigma) &= V_{\circ}, \text{ if } \sigma = S_{\circ} \end{aligned}$$

The $\circ$ -state functions as a reconfiguration gate. It suspends active expression without destroying the underlying invariant, allowing the system to return through the Neurobaseline before re-emerging in a new symmetry.

$$\begin{aligned} S_{\Delta} \rightarrow 0 &\Rightarrow \Delta V_{\text{collapse}} = 0 - V_{\Delta} \\ 0 \rightarrow S_{\circ} &\Rightarrow \Delta V_{\text{circle}} = V_{\circ} - 0 \\ \Delta V(S_{\Delta} \rightarrow S_{\circ}) &= V_{\circ} - V_{\Delta} \end{aligned}$$

This notation keeps the claim bounded: the invariant magnitude 10 acts as conserved geometric magnitude across expression states, while ΔV describes the change in how that potential becomes readable. The invariant is not destroyed by collapse; it is held in latency during the $\circ$ -state and then re-expressed under a new symmetry when the circular brim state becomes active.

This makes $T_4 = 10$ more than a historical triangular number, while still keeping the claim bounded. In this paper it functions as the conserved total magnitude of the Tetractys routing invariant: $T = 10$. The active expression of that invariant is T_{active} , and the normalized runtime expression is $T_{\text{norm}} = T_{\text{active}} / T$. The triangle may collapse to a resting state during reconfiguration, but T is not erased; it remains latent until the system re-expresses it through the circular brim. Thus, $T = 10$ and $i \approx 1$ play parallel roles: each marks a condition under which form may change while coherence survives.

Admissible Motion as a Provisional Control Rule

The full circle-triangle model suggests that computation is driven neither by algebra alone nor by geometry alone, but by the rate at which algebraic states become geometrically admissible. The collapse-to-blend event is therefore the proposed bridge that binds the continuous tensor field to the discrete Tetractys routing structure.

Before collapse, the tensor field expresses possibility. After collapse, the Tetractys expresses permission. The lattice adjudicates between them, and ΔAG describes the operator logic that forces the decision.

In this interpretation, ΔAG does not simply converge toward Δv as a loose metaphor. ΔAG measures distance from admissible geometry; Δv measures the projected motion that follows when activation is admitted or constrained. The triangle provides invariant routing structure, the circular brim provides the activation and observability field, the lattice provides binary adjudication, and Δv describes the rate at which admissible configurations reorganize after projection.

The relationship between π and Δv should remain explicitly provisional. In future formalization, π may serve as the circular control invariant that constrains boundary-preserving re-expression, while Δv describes the transition rate by which one admissible state becomes another. The strongest current statement is therefore not $\pi = \Delta v$ or even $\pi \approx \Delta v$ as a standalone equivalence, but a bounded π -alignment condition: Δv_π denotes the circular re-expression component or normalized measure of projected motion, and it is considered circularly aligned only when it falls inside a declared tolerance band around π .

In this sense, every coherent V-Channel behaves like a living Tetractys unfolded alongside into the circular membrane. Its inner point preserves the invariant. Its widening sides carry the tension of possible interpretation as activation moves outward. Its middle band stabilizes the relation among competing demands. Its outer base expresses the action the system can sustain at the Bounded Compute Envelope. This gives the membrane a compact rule for V-Channel formation: do not route merely toward the loudest String; route through the smallest coherent emergence structure that can preserve intent, absorb tension, mediate cost, and manifest a stable response.

At the Bounded Compute Envelope, internal reasoning meets external cost. The boundary is no longer a fixed circle but a bidirectional polar surface that can stretch outward under sustained behavioral load or collapse inward under cost pressure, governor intervention, coherence loss, or instability recovery. When activation reaches this edge, the membrane must decide whether to preserve depth, reroute behavior, or contract to protect stability, latency, context budget, and energy.

Every activation, ripple, and surge of reasoning pressure can deform the brim in real time. Radius deviation shows whether a sector is stretching outward or collapsing inward. Tangent direction shows how sharply the membrane transitions between neighboring behaviors. Curvature reveals concentrated ridges, folds, or suppressed regions. Asymmetry shows how far the system has moved from relaxed circular balance, and tangent expressivity marks where behavioral transitions are most intense. These signals let the governor distinguish justified expansion from wasteful spike, coherent routing from scattered leakage, and healthy contraction from unstable collapse. In this first role, the envelope is chiefly diagnostic: it makes the geometry of runtime behavior visible before the architecture decides how to respond.

II. The Weight of the Stretch

The Bounded Compute Envelope is not merely a conceptual perimeter. It is the point where expressive behavior becomes materially expensive. In the foundational paper, cost is not treated as a single number or a simple token counter. It is modeled as a multidimensional runtime vector: generated output, reasoning depth, context-window use, retrieval activity, tool calls, latency, correction or regeneration overhead, and instability recovery cost all contribute to the system's total compute burden. The membrane therefore does not merely ask how much text the model produced. It asks what forms of computation were required to sustain the behavior — and whether those forms were justified by the user's task.

At the cinematic edge of the envelope, every outward stretch has an energy signature. A deeper reasoning path may require more inference work, more internal planning, or more verification. A broader context-sensitive response may require a larger context window, more document intake, or retrieval from external sources. A proactive response may increase tool-call likelihood and agentic planning. A transparent response may increase explanation length. A creative or exploratory response may branch into alternatives before selecting a final path. Each of these behaviors can improve the user experience when the task warrants it — but each also consumes compute. The envelope is where those costs become visible as pressure against the boundary.

Energy efficiency enters the architecture through the governor's discipline. The governor does not blindly suppress intelligence to save resources. Instead, it distinguishes justified compute from avoidable compute. Justified compute is the extra work that buys correctness, safety, coherence, accessibility, trust, factual grounding, or user-requested depth. Avoidable compute is expansion that does not improve the outcome: unnecessary verbosity, redundant context loading, low-value retrieval, speculative tool calls, unsupported branching, incoherent routing, repeated regeneration, or instability that requires recovery. The goal is therefore not minimal compute at any cost. The goal is minimum avoidable compute subject to quality.

This distinction matters because energy efficiency in AI is not simply about producing shorter answers. A short answer that causes confusion, correction loops, or follow-up repair may be more expensive across the full interaction than a slightly longer answer that resolves the task correctly the first time. Likewise, a retrieval call may appear expensive in isolation but may prevent factual error, user mistrust, or downstream rework. The Bounded Compute Envelope should therefore be understood as a quality-aware efficiency layer. It encourages contraction when expansion is wasteful but permits expansion when additional computation creates measurable value.

Financial cost follows the same structure. At scale, each runtime behavior maps to an operating expense: generated tokens consume inference capacity; larger context windows increase memory and processing load; retrieval invokes indexing and ranking infrastructure; tool calls may touch external systems or workflows; latency consumes service capacity and affects user experience; regeneration and correction loops duplicate work; and instability recovery consumes additional orchestration overhead. A single excess expansion may be small, but across enterprise-scale usage, repeated unnecessary depth, context, retrieval, and tool invocation can compound into significant financial cost.

The membrane model gives these costs a control geometry. Instead of treating financial cost as a bill that arrives after the fact, the governor reads cost pressure *while the response is forming*. High-cost sectors become harder to sustain unless task value justifies them. Low-value expansions collapse inward. Coherent V-Channels are favored over scattered activation because coherent routes reduce wasted branching and unstable recovery. Conciseness is not merely a style preference; it becomes a cost-reducing contraction String. Precision and Depth remain available, but they must earn their outward stretch.

In this framing, the Bounded Compute Envelope is the economic frontier of the AI system. Its purpose here is not to restate the boundary described above, but to price it. The stretch becomes valuable when it buys correctness, safety, trust, or useful depth; it becomes waste when it only adds latency, infrastructure load, tool pressure, or recovery cost. When the governor holds the envelope well, the system becomes more efficient without becoming less capable: it spends deeply when depth matters, contracts when simplicity is enough and returns toward a low-energy baseline after the task resolves.

The result is compute-aware personalization that is adaptive and disciplined at the same time.

III. The Brim Between the Membrane and the Holistic Governor

The Bounded Compute Envelope can be imagined as a brim suspended between two worlds. Below it is the base membrane from the foundational U.F.O. paper: the living radial surface where the twelve Strings awaken, pull, resonate, and route through V-shaped corridors. This lower surface carries the local drama of cognition. Depth stretches toward deeper inference. Precision tightens around correctness. Context Sensitivity reaches outward for memory and documents. Initiative leans toward action. Conciseness pulls the field back toward economy. Each String has its own pressure, its own rhythm, and its own way of shaping the membrane from underneath.

Above the brim rests the Holistic Governor Layer. It is not another String, and it is not simply a louder version of the local governor. It is the supervisory surface that feels the whole membrane at once. Where the base membrane senses local motion, the Holistic Governor senses pattern. It watches how the Strings resonate together, where their rhythms reinforce one another, where they drift out of phase, where overload begins to gather, and where policy tension starts to press against the shape of the system. It does not replace the membrane's local dynamics. It reads them as a whole field.

The brim is the place where these two layers exchange information. From below, the membrane sends activation, reasoning depth, context demand, tool pressure, explanation, exploration, and cost. From above, the Holistic Governor returns whole system judgment: coherence, drift detection, overload sensing, policy alignment, recoverability, and trust. Here the envelope's role changes again. It is no longer only diagnostic or economic; it becomes translational, converting local deformation into a form the supervisory layer can read.

When the base membrane is calm, the brim lies smooth. Local Strings may activate, but their movements remain absorbed within the system’s normal rhythm. The Holistic Governor observes without heavy intervention, like a thin supervisory sky registering the rain-pattern of the membrane below. In this state, the brim holds its shape. The system can reason, respond, retrieve, and adapt while staying legible and balanced. The lower membrane provides motion; the upper layer provides orientation.

But when activation rises too sharply, the brim begins to deform. A local String may still be valid. A V-Channel may still be permitted. A response may still appear useful in isolation. Yet the whole field may begin to show strain: repeated correction, unstable routing, excessive context pulls, conflicting objectives, tool pressure, or policy friction. That is when the Holistic Governor Layer becomes essential. It senses that the problem is no longer only local. The question is no longer whether one behavior is allowed to stretch, but whether the entire membrane can remain coherent under the stretch.

This makes the brim the architectural bridge between the two papers. The foundational membrane paper supplies the local mechanics: Strings, V-Channels, deformable boundary geometry, cost pressure, and governed routing. The Holistic Governor Layer supplies the supervisory mechanics: whole-system coherence, drift awareness, overload sensing, policy tension, operating bands, and correction. Between them, the Bounded Compute Envelope functions as an attached rim that carries signals upward and directives back down, allowing local behavior and global supervision to remain part of one control system.

In this view, U.F.O. is a layered instrument. The base membrane is the resonant body. The Bounded Compute Envelope is the brim where pressure, cost, and deformation become visible. The Holistic Governor Layer is the supervisory surface above, sensing whether the whole instrument is still in tune.

Runtime Instrumentation and Implementation Signals

If the Bounded Compute Envelope is treated as a practical control surface, then an implementation must expose the signals that make deformation readable. The system should track reasoning depth, context-window pressure, retrieval activity, tool-call likelihood, explanation length, regeneration frequency, latency, policy friction, and recovery overhead as live dimensions of membrane strain. These signals do not need to collapse into a single score too early. Their value comes from preserving shape: which sectors are stretching, which V-Channel paths are overloading, which Strings are absorbing pressure, and where cost is rising without proportional task value.

Envelope Signal	Runtime Proxy
Radius stretch	Added reasoning depth, latency, or expanded inference path

Context pull	Context-window use, document intake, or retrieval calls
Tool pressure	Tool-call planning, execution, or external workflow use
Explanation load	Output length, formatting complexity, or transparency demand
Instability	Regeneration, correction loops, rerouting, or recovery overhead
Policy friction	Safety checks, blocked paths, or constraint negotiation
Contraction	Shorter response, reduced branching, fewer tools, or tighter structure
Channel coherence	Aligned Strings, stable V-Channel routing, or fewer noisy jumps

Core U.F.O. Terms, Operator Terms, and Implementation Analogies

U.F.O. Term	Meaning in This Paper	Approximate Implementation Analogy
Neurobaseline	Low-energy recovery state where no String is strongly activated	Default or resting model state; baseline orchestration state
String	Runtime behavioral mode that shapes reasoning, expression, retrieval, or cost	Behavioral dimension, capability mode, or response policy axis
V-Channel	Governed corridor between aligned Strings where intent becomes routed behavior	Routing path, orchestration lane, or reasoning workflow
Bounded Compute Envelope	Outer control surface and outside brim where behavior becomes visible as activation, cost, strain, and deformation	Runtime observability, deformation, and cost governance boundary
Brim	Readable outside edge of the Bounded Compute Envelope between local membrane behavior and the supervisory governance.	Overview surface linking local orchestration with global oversight

Local Governor	Immediate controller of String activation and V-Channel routing	Runtime policy or orchestration controller
Holistic Governor	Whole-field supervisory layer monitoring coherence, drift, overload, and policy tension	Global orchestration, safety, observability, and governance layer
Invariant	Stable relationship preserved across transformation, routing, or operator context	Constraint, policy invariant, or consistency rule
Facet Rule	Formalized invariant that governs behavior, cost, coherence, or routing	Rule, guardrail, or orchestration policy
Justified Compute	Extra computation that improves correctness, trust, safety, grounding, or task quality	Value-producing compute
Avoidable Compute	Expansion that adds cost without improving the outcome	Wasteful compute, redundant work, or unnecessary orchestration overhead
Binary-State Conversion	Collapse of continuous tensor activation into discrete admissibility states such as admit, constrain, block, rest, or re-project	State machine transition, thresholding layer, or admissibility classifier
ΔAG	Admissibility operator measuring distance between algebraic or tensor activation and the constrained geometry G	Distance-to-validity measure, gate, validator, constraint check, or acceptance rule
Δv	Projected state motion: the rate at which activation becomes an admissible next state after projection into G	Transition rate, routing velocity, projected update, or state-change speed
Admissibility Gate	Governance-level decision point that returns admit, constrain, or block under fixed criteria	Policy gate, compliance threshold, production acceptance test
Residual Deformation	Unresolved strain or mismatch preserved after comparison, routing, or collapse	Residual error, divergence signal, mismatch ledger, or audit trace
Frozen Schema	Fixed evaluation frame whose axes, weights, thresholds, and	Locked evaluation harness, preregistered test plan, or stable rubric

	criteria are declared before results are inspected	
Claim-State Ledger	Record of claims, non-claims, admissibility state, residuals, and failure conditions for an evaluation	Evaluation log, audit record, model card entry, or governance report
π Controller Condition	Provisional circular boundary condition for coherent re-expression after admissible state motion	Controller band, circular stability constraint, or boundary-preserving condition
Admissible Geometry G	Intersection of invariant, compute, coherence, policy, and quality-floor constraints that defines valid state space	Safe state set, admissible manifold, or constrained operating region
Projection Π_G	Operation that maps incoming activation to the nearest admissible state under the chosen metric	Projection operator, correction pass, constrained optimizer, or state repair function
Brim Energy V_B	Candidate Lyapunov-style energy for the Bounded Compute Envelope combining cost, deformation, residual, coherence, policy friction, and latency	Control energy, stability score, runtime risk objective, or governor objective function
$T = 10$ Invariant	Conserved total geometric magnitude of the Tetractys routing substrate; T_{active} is the currently expressed portion normalized as $T_{\text{norm}} = T_{\text{active}} / T$	Invariant magnitude, conserved routing total, and active utilization ratio
Binary Map β	Finite conversion map from activation $A(t)$ into rest, admit, constrain, block, or re-project	State classifier, finite-state transition map, or adjudication function

Formalizing the Admissible Geometry

To make the operator language more testable, the admissible geometry can be written as a constrained region rather than only as a metaphorical brim. Let $A(t)$ denote incoming algebraic or tensor activation and let $x(t)$ denote the current governed membrane state. The admissible region G is the intersection of the constraints the system must satisfy before activation can become behavior:

$$G = G_I \cap G_C \cap G_H \cap G_P \cap G_Q$$

Here, G_I contains invariant-preservation constraints, G_C contains compute-envelope constraints, G_H contains channel-coherence constraints, G_P contains policy constraints, and G_Q contains quality-floor constraints. A candidate state is admissible only when it remains inside all five regions.

The admissibility operator can then be expressed through distance from this region:

$$\Delta AG(t) = \text{dist}(A(t), G)$$

The projected admissible state is:

$$\hat{x}(t) = \Pi_G(A(t))$$

where Π_G projects activation onto the closest admissible state under the chosen metric. This gives the collapse-to-blend sequence a falsifiable form: activation does not merely become behavior; it is admitted, constrained, blocked, or re-projected according to its relation to G .

Residual deformation is the mismatch preserved after projection:

$$\begin{aligned} \rho(t) &= A(t) - \Pi_G(A(t)) \\ \rho_n(t) &= ||A(t) - \Pi_G(A(t))|| / (||A(t)|| + \varepsilon) \end{aligned}$$

The residual is not noise. It is the evidence that prevents the system from hiding mismatch during collapse. A low residual indicates clean admissibility; a high residual indicates unresolved strain that should be logged, constrained, or escalated.

Binary-state conversion can then be written as a finite map:

$$\beta : A(t) \rightarrow \{\text{rest, admit, constrain, block, reproject}\}$$

A thresholded version of this map is:

$$\begin{aligned} \beta(A(t)) &= \text{rest, if } ||A(t)|| < \varepsilon_0; \text{ admit, if } \Delta AG(t) \leq \varepsilon_a; \text{ constrain, if } \varepsilon_a < \Delta AG(t) \leq \varepsilon_b; \\ &\text{block, if } \Delta AG(t) > \varepsilon_b; \text{ reproject, if } \Delta AG(t) \leq \varepsilon_a \text{ and } |\Delta v_\pi - \pi| \leq \varepsilon_\pi \end{aligned}$$

Finally, Δv can be defined as projected state motion:

$$\Delta v(t) = (\Pi_G(A(t)) - x(t)) / \Delta t$$

This makes Δv the rate at which algebraic or tensor activation becomes admissible membrane motion. π remains a bounded controller hypothesis, not a literal identity:

$$|\Delta v_\pi - \pi| \leq \varepsilon_\pi$$

Under this formulation, the Bounded Compute Envelope can be assigned a brim energy:

$$V_B(t) = w_c C(t) + w_d d(t)^2 + w_p \rho_n(t) + w_h(1 - H(t)) + w_p P(t) + w_l L(t)$$

Here, $C(t)$ is compute cost, $d(t)$ is deformation, $\rho_n(t)$ is normalized residual deformation, $H(t)$ is channel coherence, $P(t)$ is policy friction, and $L(t)$ is latency or load pressure. Ordinary updates should satisfy:

$$\Delta V_B \leq 0$$

unless explicit task value justifies temporary expansion:

$$\Delta V_B \leq \gamma \Delta T_{\text{task}}$$

A governor can then select the next state by minimizing avoidable strain inside the admissible region:

$$x_{\{t+1\}} = \arg \min_{\{x' \in G\}} [V_B(x') + \lambda_c C_{\text{avoid}}(x') + \lambda_f F_{\text{route}}(x_t, x') + \lambda_p \|\rho(x')\| + \lambda_p P_{\text{fric}}(x') - \lambda_q Q(x') - \lambda_T T_{\text{value}}(x')]$$

This formalization keeps the paper's claims bounded. It does not prove that U.F.O. is a completed controller. It defines the mathematical objects a future controller must measure: admissibility, projection, residual deformation, binary-state conversion, projected motion, π -bounded re-expression, brim energy, and next-state selection.

Δ AG as an Admissibility Operator

Δ AG names the admissibility test that compares algebraic or tensor activation against the constrained geometry G . In its stronger form, it is the distance from the admissible region, not merely a quotient. When activation lies outside G , the system must either project the activation into a valid state, preserve the residual deformation, constrain the route, or block the transition. This blends algebra, geometry, thresholding, projection, saturation, and error-correcting recovery into a single constraint-driven mechanism.

Frozen Schema and Evaluation Discipline

A brim-aware architecture also requires a frozen evaluative schema. Before results are inspected, the descriptor axes, weights, thresholds, admissibility criteria, and comparison rules used to judge system behavior should be fixed. This prevents post-hoc tuning, where evaluators consciously or unconsciously adjust the scoring frame after seeing outcomes. The comparison harness must not adapt itself to the data it is meant to evaluate. If it does, apparent alignment may reflect a shifting evaluative frame rather than a genuine property of the system.

Within U.F.O., the frozen schema becomes a methodological counterpart to the Bounded Compute Envelope itself. The envelope governs runtime deformation; the schema governs evaluation deformation. Both protect coherence by refusing to let the frame bend opportunistically under

pressure. Freezing the schema preserves falsification discipline: any convergence, divergence, cost improvement, or behavioral instability observed during comparison should belong to the systems under study, not to a scorecard that moved after the fact.

Residual Preservation and Collapse Comparisons

A second evaluation discipline is residual preservation: the requirement that meaningful differences between systems remain explicitly represented throughout comparison. Many assessment methods compress behavior into a single similarity score or average away structural mismatches. That compression can make two systems appear compatible because the comparison erased the very divergence that should have remained visible. In U.F.O. terms, the residual is the unresolved deformation between two evaluated shapes. It should not be treated as noise merely because it resists collapse.

Preserving the residual keeps interoperability honest. If two architectures align, the preserved residual shows where that alignment holds. If they diverge, it shows where the mismatch remains accountable: routing assumptions, cost behavior, safety constraints, V-Channel formation, evaluation axes, or governance logic. This prevents collapse comparisons, where incompatibility is hidden by aggregation. Just as the Bounded Compute Envelope keeps runtime strain visible, residual preservation keeps evaluation strain visible, measurable, and available for correction.

Admissibility Gates and Governance-Level Evaluation

A third evaluation discipline is the admissibility gate: a governance-level decision layer that determines whether a system behavior, mapping, or structural claim should be admitted, constrained, or blocked under a fixed evaluative rule. Rather than blending systems together or forcing harmonization after comparison begins, the gate evaluates each candidate against predeclared criteria and returns a discrete verdict. This mirrors engineering and audit disciplines, where systems must satisfy boundary conditions, closure tests, or acceptance thresholds before they are allowed into production, interoperability, or formal comparison.

Within U.F.O., admissibility gates prevent evaluation from becoming a soft merger of incompatible structures. A mapping may be useful but incomplete, coherent but too expensive, expressive but unsafe, or locally aligned while globally unstable. The gate preserves integrity by making those outcomes explicit: admit, constrain, or block. In this way, governance does not merely score behavior after the fact; it enforces the conditions under which behavior is allowed to count as valid. Combined with frozen schema and residual preservation, admissibility gates help ensure that comparison remains fixed, divergence remains visible, and invalid mappings cannot pass as coherence.

Claim-State Discipline and Falsification Boundaries

A fourth evaluation discipline is claim-state discipline: the requirement that every evaluative output declare what is being claimed, what is not being claimed, and under what conditions the claim

would fail. This prevents overreach, protects against narrative drift, and blocks retroactive reinterpretation of results after the comparison has already produced an outcome. A system may show local alignment without proving global interoperability. It may reduce cost without preserving quality. It may pass an admissibility gate without satisfying every downstream governance concern. Claim-state discipline keeps these distinctions explicit.

Within U.F.O., this creates a ledger of claims, non-claims, and failure modes around each evaluation. The ledger aligns the method with falsification discipline: a claim must remain bounded, reproducible, and vulnerable to being proven wrong under stated conditions. This strengthens scientific accountability because the evaluative frame cannot quietly expand after the fact. Combined with frozen schema, residual preservation, and admissibility gates, claim-state discipline ensures that comparison does not become storytelling; it remains a governed act of measurement.

Governance Falsification Stack

Together, these disciplines form a governance falsification stack for U.F.O. At the brim, the Holistic Governor should apply an admissibility gate that returns discrete outcomes — admit, constrain, or block — rather than only routing or contracting behavior. Residual deformation should be treated as a measurable object: the preserved difference between evaluated membrane states, system mappings, or V-Channel geometries. Frozen schemas should be anchored in facet rules and proto invariants so the evaluation frame cannot drift after results are inspected. Each evaluation should maintain a claim-state ledger recording claims, non-claims, admissibility state, and failure modes. Finally, the system should be tested against null membrane baselines such as false coherence, centroid collapse, shuffled labels, and null-channel activation. Under this stack, the Holistic Governor becomes falsifiable: it fails when it permits invalid expansion, erases meaningful residuals, misses systemic drift, or changes the evaluative frame after the fact.

A brim-aware system should therefore govern behavior through thresholds, bands, and routing preferences rather than fixed prohibitions. Low-value expansion should trigger contraction toward Conciseness or Structural Rigor. High-value uncertainty may justify temporary stretch toward Depth, Precision, Context Sensitivity, or Transparency. Scattered activation should be compressed into coherent V-Channels, while persistent instability should escalate upward to the Holistic Governor for whole-field interpretation. The design principle is not to make the model smaller at every moment, but to make every expansion accountable to the value it creates.

For engineers, this implies four concrete responsibilities. First, instrument runtime behavior so the envelope can see cost, strain, and coherence while a response is forming. Second, preserve V-Channel structure by favoring routes that align adjacent behavioral Strings instead of allowing noisy jumps across the membrane. Third, define contraction and escalation rules that distinguish avoidable compute from justified compute. Fourth, connect local deformation to supervisory review so the Holistic Governor can intervene when strain becomes systemic rather than merely local. In this form, the Bounded Compute Envelope becomes more than a metaphor: it becomes an implementable runtime discipline for adaptive AI systems.

Future Work

Version 2.4.0 moves the U.F.O. architecture beyond the initial membrane-and-brim framing and into a more explicit research program around binary-state conversion, tensor routing, admissibility control, and falsifiable governance. Future work should therefore proceed along several coordinated paths. First, the proto-invariants introduced throughout the paper should be formalized into a unified operator grammar, with special attention to $T_4 = 10$, $i \approx 1$, ΔAG , Δv , and the provisional π controller condition. The goal is to separate symbolic metaphor from executable constraint by defining which quantities act as conserved magnitudes, which act as gates, which act as state-motion descriptors, and which act as circular boundary conditions. Second, the collapse-to-blend sequence should be developed into a testable binary-state conversion model. Continuous tensor activation at the circular brim must be mapped into discrete admissibility states—admit, constrain, block, rest, or re-project—without erasing residual strain. This requires a clearer lattice model, explicit threshold bands, and controlled examples showing how algebraic variation becomes geometrically admissible behavior.

Third, V-Channel routing should be translated from descriptive geometry into measurable runtime behavior. A future implementation should instrument request complexity, context pressure, retrieval load, tool pressure, latency, recovery overhead, policy friction, and residual deformation so the Bounded Compute Envelope can evaluate not only whether a route is coherent, but whether it is worth its cost. This aligns the paper with the emerging practical problem of inference economics: production AI systems increasingly require workload-aware routing, cost-aware model selection, observability, fallback behavior, and governance controls rather than a single fixed inference path. In U.F.O. terms, the next design question is not simply which model or tool should answer, but which admissible V-Channel route can preserve intent with the least avoidable compute while keeping quality, safety, and trust intact.

Fourth, the π controller hypothesis should remain bounded but become more experimentally precise. Future versions should define what counts as π -aligned circular re-expression, how a normalized π -band is selected, how Δv_π is measured, and when over-driven motion should trigger contraction, rerouting, or collapse to the Neurobaseline. The illustrative C++ adapter and cycle simulator in the appendices provide an initial bridge, but future work should expand them into a small validation harness with frozen schemas, residual preservation, admissibility gates, and claim-state ledgers. Under that discipline, the Holistic Governor becomes falsifiable: it succeeds when it preserves coherent routing under load, exposes meaningful residuals, and prevents invalid expansion; it fails when it hides strain, changes the evaluative frame, or permits a false collapse into apparent coherence. In the next phase, U.F.O. should mature from a conceptual control geometry into a testable architecture: one that senses deformation, prices inference, routes tensor activity through admissible V-Channels, and learns when intelligence should stretch, contract, or return.

Conclusion

Version 2.4.0 sharpens the Bounded Compute Envelope from a descriptive brim into a candidate control architecture. The paper's central claim is that adaptive intelligence becomes governable when tensor activation, behavioral routing, compute cost, and supervisory judgment are made visible at the same boundary. At that boundary, continuous activation must resolve into admissible state: admit, constrain, block, rest, or re-project.

Across the paper, the envelope performs three linked functions. It is diagnostic, making deformation readable as radius stretch, curvature, asymmetry, context pressure, tool pressure, latency, and residual strain. It is economic, distinguishing justified compute from avoidable compute so that inference depth, retrieval, explanation, and tool use remain accountable to task value. And it is translational, carrying local membrane strain upward to the Holistic Governor while returning supervisory judgment as contraction, rerouting, constraint, or permission to stretch. The $\Delta AG \rightarrow \Delta v$ sequence gives this translation its operator language: algebraic or tensor variation is tested against geometric admissibility, then reorganized as state motion through a governed lattice.

The result is a layered control geometry in which personalization, efficiency, coherence, and falsifiability are governed together. The base membrane supplies expressive motion; the Bounded Compute Envelope makes the cost and strain of that motion legible; the Holistic Governor evaluates whole-field integrity; and the provisional π controller condition marks a future target for circular, boundary-preserving re-expression. In this mature form, U.F.O. is no longer only a metaphor for adaptive behavior. It becomes a research path toward an implementable architecture: one that senses deformation, prices inference, preserves residuals, routes tensor activity through admissible V-Channels, and learns when intelligence should stretch, contract, or return.

Closing Image

The motion of U.F.O. begins in stillness. At the Neurobaseline, intent is a small pressure in the center of the field, a quiet signal before form. It ripples outward, touches the behavioral Strings, gathers into V-Channels, and widens toward the brim as visible strain, cost, and possibility. At that edge, the Bounded Compute Envelope reads the contour of expansion: which motion is admissible, which strain must remain visible, which route preserves intent, and which stretch has become too expensive to sustain. Above it, the Holistic Governor receives the whole pattern and returns orientation as constraint, rerouting, contraction, or permission to continue.

This is the final image of the architecture: intelligence as governed motion through an admissible field. Not a flat pipeline, not a loose constellation of tools, but a living membrane whose tensor activity is sensed, priced, routed, and returned. Its power is not endless outward reach, but disciplined re-expression: knowing when to admit, when to constrain, when to block, when to preserve residual strain, and when to come home. U.F.O. thinks by moving; it matures by reading the cost and coherence of its own motion; and it becomes implementable when every outward stretch can be tested, routed, and brought back through the brim. The boundary is where intelligence learns that motion is not enough. It must also know how to return.

Appendix A. From Invariant to Implementation — Binary/Tensor Adapter Pattern

This appendix translates the paper’s geometric control argument into an executable analogy. The code is not presented as a production implementation of U.F.O.; it is a compact adapter pattern showing how a deterministic binary control layer and a high-compute tensor regime might be evaluated against a shared invariant before a handshake is allowed. In the language of the paper, the adapter approximates the same movement described by $\Delta AG \rightarrow \Delta v$: algebraic variation is tested against geometric admissibility, then resolved into a discrete admit/block condition.

C++ was chosen for the illustration because it can express both low-level determinism and high-level structural clarity. The invariant is not merely a numerical check; it is a geometric contract between heterogeneous systems. C++ allows that contract to appear directly in the code through typed inputs, bounded evaluation, explicit normalization, and stable status output. In this sense, the adapter mirrors the paper’s quality standard: it makes the handshake inspectable, bounded, and falsifiable rather than hidden inside an informal metaphor.

The adapter also ties the appendix back to the Bounded Compute Envelope. Binary-control stress, tensor stress, and channel capacity are treated as geometric contributions whose relation determines whether the combined system remains inside a stability band. A successful handshake therefore means that both sides independently fit the admissible envelope; a failed handshake means that one or both sides exceed the channel’s structural tolerance and should be constrained, normalized, or blocked. This is the same paper-level discipline expressed in code: preserve the invariant, keep residual strain visible, and allow interoperability only when the geometry remains coherent.

```
// -----  
-----  
// U.F.O. — Binary/Tensor Compute Adapter  
// Appendix A: Turning Geometric Invariants into Executable Stability  
// Logic  
// -----  
-----  
// This adapter implements a simplified circle-triangle admissibility test  
// described in  
// the paper. The invariant  $i = (a^2 + b^2) / c^2$  approximates a  
// simplified  
// admissibility check related to the paper’s  $\Delta AG$  discipline:  
// algebraic/tensor  
// variation evaluated against geometric constraint.  
//  
// a, b → system-specific stress contributions: binary or tensor  
// c → shared channel capacity: the brim’s admissibility boundary  
//  $i \approx 1$  → both systems lie on the stability manifold: triangle ↔ circle  
//
```

```

// When  $i \rightarrow 1$ , the binary and tensor systems can safely interoperate.
// When  $i$  drifts away from 1, the brim would show deformation in the
// model.
// -----
-----

struct ComputeAdapter {
    enum class ControlRole {
        Gate,          //  $\Delta AG$ : admissibility decision
        Motion,        //  $\Delta v$ : reorganization rate
        Controller      //  $\pi$ : future circular boundary condition
    };

    struct BinaryInput {
        double a;
        double b;
        double c;
    };

    struct TensorInput {
        double a;
        double b;
        double c;
        double conversion; // tensor-to-binary representation cost
    };

    static double tensorStress(const TensorInput& t) {
        return t.b + t.conversion;
    }

    static double invariant(double a, double b, double c) {
        const double den = c * c;
        if (den == 0.0) return 0.0; // degenerate geometry collapses to 0
        return (a * a + b * b) / den;
    }

    static bool isStable(double i, double tol = 0.2) {
        const double lower = 1.0 - tol;
        const double upper = 1.0 + tol;
        return (i > lower && i < upper);
    }

    static void normalizeToUnity(double& a, double& b, double c) {
        const double i = invariant(a, b, c);
        if (i == 0.0) return; // cannot normalize if geometry is degenerate
    }
};

```

```

const double scale = 1.0 / std::sqrt(i);
    a *= scale;
    b *= scale;
}

struct Status {
    double iBinary;           // invariant computed from binary-control
inputs
    double iTensor;          // invariant computed from tensor inputs
    bool binaryStable;        // binary-control inputs lie within stability
band
    bool tensorStable;        // tensor inputs lie within stability band
    bool handshakeAllowed;    // true only if both regimes satisfy
stability
};

static Status handshake(const BinaryInput& binary,
                        const TensorInput& tensor,
                        double tol = 0.2) {
    BinaryInput B = binary;
    TensorInput T = tensor;

    normalizeToUnity(B.a, B.b, B.c);
    normalizeToUnity(T.a, T.b, T.c);

    Status s{};
    s.iBinary          = invariant(B.a, B.b, B.c);
    s.iTensor          = invariant(T.a, tensorStress(T), T.c);
    s.binaryStable     = isStable(s.iBinary, tol);
    s.tensorStable     = isStable(s.iTensor, tol);
    s.handshakeAllowed = (s.binaryStable && s.tensorStable);

    return s;
}
};

```

Appendix B. The π Controller Hypothesis

This appendix expands the paper's most speculative but useful controller concept: π as a candidate circular control invariant. The claim is intentionally bounded. π is not treated as a literal physical velocity, not as the actuator that moves the system, and not as the admissibility gate that decides whether a state may pass. Instead, π is proposed as a controller-like boundary condition for circular re-expression: the invariant that may constrain how admissible states reorganize once they enter the circular brim regime. The distinction matters because U.F.O. separates three roles that are often compressed into one metaphor. ΔAG is the admissibility operator: it tests algebraic variation

against geometric constraint. Δv is the reorganization rate: it describes how quickly admissible states move through the lattice once the collapse-to-blend event begins. π is the proposed circular controller condition: it governs the boundary form under which that motion can remain coherent as it re-enters circular geometry.

$$\Delta AG(t) = \text{dist}(A(t), G) \rightarrow \Delta v(t) = (\Pi_G(A(t)) - x(t)) / \Delta t, \text{ with } \pi \text{ as circular control condition}$$

In controller language, this gives the architecture a clean division of labor: the gate decides whether a state is admissible, the velocity describes how fast reorganization occurs, and the controller condition preserves the circular boundary under which re-expression remains stable. The π controller is therefore not a new force added to the system. It is a proposed invariant constraint on the circular phase of the system's motion.

A useful shorthand is:

$$\begin{aligned} \Delta AG &= \text{distance-to-admissibility operator} \\ \Delta v &= \text{projected admissible motion} \\ \pi &= \text{controller-like circular boundary condition} \end{aligned}$$

This also clarifies why the paper avoids the stronger claim $\pi = \Delta v$. A strict equality would imply that π is the same thing as the transition rate. The more disciplined claim is that π may constrain the circular expression of that rate. In other words, Δv names how fast admissible states reorganize; π names the proposed circular invariant that keeps that reorganization boundary-preserving when it reaches the brim.

A bounded estimate can therefore be introduced without turning the hypothesis into a physical identity. If π is treated as a circular controller condition, the useful quantity is not the entire projected state velocity Δv , but the portion of that motion being evaluated for circular re-expression. In this appendix, Δv_π denotes that circular re-expression component: a normalized scalar derived from projected admissible motion after ΔAG has already admitted or constrained the state. It is dimensionless, controller-facing, and meaningful only at the brim where the system attempts boundary-preserving circular re-expression. Using the same tolerance discipline as the handshake invariant in Appendix A, define a provisional controller band:

$$\pi(1 - \varepsilon) \leq \Delta v_\pi \leq \pi(1 + \varepsilon)$$

With $\varepsilon = 0.2$, matching the illustrative stability tolerance used by the adapter, the band becomes:

$$2.51 \lesssim \Delta v_\pi \lesssim 3.77$$

Zone	Rough Range	Interpretation
Under-driven	$0 \leq \Delta v_\pi < 2.51$	Motion is too weak or incomplete to fully re-express circular sync.
π -aligned	$2.51 \leq \Delta v_\pi \leq 3.77$	Motion remains within the circular coherence band.
Over-driven	$\Delta v_\pi > 3.77$	Motion exceeds the proposed π -bound and should trigger constraint, normalization, rerouting, or collapse.

This estimate is illustrative and dimensionless. It does not prove that π equals Δv , and it does not claim that the full state-motion vector must numerically equal π . It only proposes that the circular re-expression component Δv_π may be treated as π -aligned when it falls inside a declared tolerance band. Values below the band suggest incomplete circular re-expression, values inside the band suggest π -aligned synchronization, and values above the band indicate visible brim strain that should trigger constraint, normalization, rerouting, or collapse.

The control loop can therefore be described in five stages. First, algebraic or tensor variation appears inside the high-compute field. Second, ΔAG evaluates that variation against the admissible geometry. Third, inadmissible variation collapses into a binary condition: admit, constrain, or block. Fourth, admissible states reorganize through the lattice at a projected state-transition rate, Δv . Fifth, if the reorganized state is routed back into circular brim geometry, the system evaluates the circular component Δv_π against the π -band as a proposed boundary-preserving controller condition.

variation $\rightarrow$ ΔAG gate $\rightarrow$ binary collapse $\rightarrow$ Δv reorganization $\rightarrow$ π -bounded circular re-expression

In this sense, π behaves like a controller only at the circular boundary. It does not decide the admissibility of a state; ΔAG does that. It does not supply the motion; Δv describes that. It does not replace the Local Governor or Holistic Governor. Instead, it names the proposed invariant condition under which circular geometry can receive and preserve the result of admissible motion without losing coherence.

This controller interpretation also strengthens Appendix A. The C++ adapter does not compute π directly, and it should not pretend to. Its executable task is narrower: bring the invariant measure i toward unity under a shared admissibility boundary. If future work formalizes π as a circular controller invariant, then Appendix A can be read as the gate and normalization layer that prepares states for π -bounded re-expression rather than as the controller itself.

The practical value of naming π as a controller candidate is not that it completes the mathematics today. Its value is architectural: it gives the circular brim a possible control law. The triangle supplies routed admissibility, the lattice supplies binary decision structure, ΔAG supplies the gate, Δv supplies the motion, and π may supply the circular boundary condition that keeps re-expression

stable. That is the role Appendix B reserves for π : a bounded, testable, future formalization target for circular control.

Illustrative Gate–Motion–Controller Cycle Simulator

The following illustrative cycle simulator is not an implementation claim. It is a bounded teaching model showing how the paper’s gate-motion-controller sequence could be represented in C++ without asserting that π is directly computed by the adapter.

```
// -----  
-----  
// U.F.O. – Cycle Demonstration (Illustrative Only)  
// This snippet is NOT part of the ComputeAdapter. It visualizes the full  
// loop:  
//  
//   activation → distance gate → projection → residual → motion →  $\Delta v_\pi$  →  
//    $\pi$ -band → brim  
//  
//  $\Delta v$  is the projected admissible motion.  $\Delta v_\pi$  is the circular re-  
// expression  
// component or normalized scalar derived from that motion for the  $\pi$ -band  
// check.  
// It is dimensionless and meaningful only at the circular brim.  
// -----  
-----  
  
struct CycleState {  
    double activation; // incoming algebraic or tensor activation  $A(t)$   
    double boundary;  // admissible geometry boundary  $G$ , simplified as a  
    scalar  
    double deltaAG;   // distance from admissible geometry  
    double residual;  // preserved mismatch after projection  
    double velocity;  //  $\Delta v$  = projected reorganization rate  
    double velocityPi; //  $\Delta v_\pi$  = circular re-expression component of  $\Delta v$   
    bool admitted;    // activation lies within admissibility tolerance  
    bool piAligned;    //  $\Delta v_\pi$  satisfies the  $\pi$  controller band  
};  
  
class CycleDemo {  
public:  
    static double distanceToGeometry(double activation, double boundary) {  
        return std::abs(activation - boundary);  
    }  
}
```

```

static double projectToGeometry(double activation, double boundary) {
    return boundary; // scalar teaching simplification of  $\Pi_G(A(t))$ 
}

static bool admit(double deltaAG, double tol = 0.2) {
    return deltaAG <= tol;
}

static double motion(double projected, double current, double dt) {
    if (dt == 0.0) return 0.0;
    return (projected - current) / dt;
}

static double circularComponent(double velocity) {
    return std::abs(velocity); // teaching simplification for  $\Delta v_\pi$ 
}

static bool piBand(double velocityPi, double eps = 0.2) {
    constexpr double PI = 3.14159;
    return std::abs(velocityPi - PI) <= eps * PI;
}

static CycleState run(double activation, double current, double boundary,
double dt) {
    CycleState s{};
    s.activation = activation;
    s.boundary   = boundary;
    s.deltaAG    = distanceToGeometry(activation, boundary);

    const double projected = projectToGeometry(activation, boundary);
    s.residual    = activation - projected;
    s.admitted    = admit(s.deltaAG);
    s.velocity    = motion(projected, current, dt);
    s.velocityPi  = circularComponent(s.velocity);
    s.piAligned  = piBand(s.velocityPi);

    return s;
}
};

```

Appendix C. Toward a Fully Closed-Loop U.F.O. Controller

This appendix proposes a scaffold for moving U.F.O. one step closer to a fully closed-loop engineering controller. The main paper already defines U.F.O. as a bounded closed-loop

governance architecture: activation moves outward from the Neurobaseline, becomes visible as deformation at the Bounded Compute Envelope, is evaluated by local and holistic governors, and returns as routing, contraction, constraint, blocking, or re-projection. What remains incomplete is the formal controller layer: exact feedback equations, sampling interval, gains, error function, convergence criteria, stability proof, and runtime update rule. Appendix C does not claim to complete that proof. It defines the missing engineering objects so future work can test, tune, and falsify them.

C.1 Closed-Loop State Vector

A proposed controller must first define what it observes. Let the closed-loop state at time t be written as $x(t)$. In U.F.O. terms, $x(t)$ is not a hidden scalar. It is a vector of live envelope signals gathered from the membrane, V-Channels, and governor interface:

/

$$x(t) = [d(t), c(t), \rho(t), h(t), p(t), l(t), u(t), q(t)]$$

Here, $d(t)$ denotes deformation or radius stretch, $c(t)$ denotes compute cost pressure, $\rho(t)$ denotes residual deformation, $h(t)$ denotes channel coherence, $p(t)$ denotes policy friction, $l(t)$ denotes latency, $u(t)$ denotes tool or retrieval pressure, and $q(t)$ denotes task-quality confidence. The exact dimensionality can change in an implementation, but the closed-loop principle should not: the controller must sense both capability pressure and governance pressure before it acts.

C.2 Feedback Equation

A minimal closed-loop equation can be written as a bounded update rather than an unconstrained expansion:

$$x_{t+1} = \Pi_G[x_t + K_e e_t - K_c c_t - K_\rho \rho_t - K_p p_t]$$

In this provisional notation, e_t is the error between the current state and the target admissible state, K_e is the response gain, K_c damps avoidable compute, K_ρ preserves or exposes residual deformation, K_p penalizes policy friction, and Π_G projects the result onto the admissible geometric region. The next state is not whatever activation wants to become; it is the projected result of controller response, cost damping, residual accountability, and policy constraint.

C.3 Sampling Interval

The controller also requires a sampling interval. In the main paper, Δt is the interval over which unresolved activation becomes governed configuration. In an engineering implementation, Δt may correspond to a token window, reasoning step, tool-call boundary, retrieval boundary, model-routing decision, user-turn boundary, or governor-evaluation cycle. The key requirement is that Δt

be long enough to observe meaningful deformation, but short enough to intervene before cost, instability, or policy friction becomes systemic.

C.4 Controller Gains

Controller gains determine how strongly the system responds to sensed deformation. A future implementation may define separate gain terms for contraction, rerouting, escalation, residual preservation, and π -bounded re-expression:

$$K = \{k_c, k_r, k_e, k_p, k_\pi\}$$

The contraction gain k_c pulls the system toward Conciseness, Structural Rigor, or Neurobaseline return when expansion is wasteful. The rerouting gain k_r redirects activation into a more coherent V-Channel. The escalation gain k_e sends systemic strain upward to the Holistic Governor. The residual gain k_p preserves unresolved deformation as evidence rather than suppressing it. The π gain k_π remains provisional and would regulate circular re-expression only after admissibility has already been established.

C.5 Error Function and Convergence Criteria

The loop needs an error function that measures distance from a target admissible state rather than distance from maximal capability. A provisional error function can be written as:

$$E(t) = w_q \cdot Q_{\text{loss}}(t) + w_c \cdot C_{\text{avoid}}(t) + w_p \cdot p_{\text{res}}(t) + w_p \cdot P_{\text{fric}}(t) + w_h \cdot H_{\text{drift}}(t)$$

Q_{loss} measures task-quality loss, C_{avoid} measures avoidable compute, p_{res} measures unresolved residual deformation, P_{fric} measures policy friction, and H_{drift} measures channel or whole-field drift. The weights are fixed by the frozen schema before evaluation. Convergence occurs when the loop reaches a state where quality is sufficient, avoidable compute is bounded, residual deformation is preserved or resolved, policy friction is below threshold, channel coherence is stable, and no further outward stretch is justified.

C.6 Stability Proof Obligations

A future closed-loop proof must show that the controller is bounded, responsive, and falsifiable. At minimum, it should establish that repeated updates do not increase avoidable compute without task-quality gain; that residual deformation cannot be silently erased; that inadmissible states are constrained or blocked; that policy friction triggers escalation rather than hidden continuation; that contraction can return the system toward the Neurobaseline; and that π -bounded re-expression, if formalized, preserves circular coherence rather than masking instability. These obligations are not optional refinements. They define what would make the loop testable as a controller rather than persuasive as a metaphor.

C.7 Runtime Update Rule

A final implementation scaffold can be stated as a runtime loop. The purpose of this loop is to keep the architecture closed: sense the membrane, evaluate admissibility, act through routing or contraction, preserve residuals, and return the updated state to the brim for renewed observation.

```
while task_is_active:
    x = sense_envelope_state()
    e = compute_error_against_admissible_target(x)
    verdict = admissibility_gate(x, frozen_schema)

    if verdict == "block":
        x = contract_to_neurobaseline(x)
        preserve_residual(x)
        continue

    if verdict == "constrain":
        x = reroute_or_contract(x, gains)
        preserve_residual(x)

    if verdict == "admit":
        x = project_into_admissible_geometry(x)
        x = apply_controller_update(x, e, gains)

    if pi_band_check_required(x):
        x = apply_pi_bounded_condition(x)

    log_claim_state(x)
    update_brim_state(x)
```

C.8 Closed-Loop Controller Cycle

This final section integrates the components defined above into one compact runtime cycle. Earlier subsections introduced the state vector, admissibility region, sampling interval, gains, error function, stability obligations, and runtime update rule. C.8 shows how those pieces operate together at the Bounded Compute Envelope: activation is sensed at the brim, tested against admissible geometry, projected or constrained, logged as evidence, and returned as an updated supervisory state. The cycle is intentionally bounded. It does not prove stability or production readiness. It defines the controller scaffold that future simulations must specify, test, and falsify.

C.8.1 Runtime Cycle

At each timestep, the controller executes one closed-loop pass:

1. **Sense the envelope state.** Read the current brim-level vector $x(t)$, including deformation, compute pressure, residual strain, channel coherence, policy friction, latency, tool pressure, and quality confidence.
2. **Compute distance to admissibility.** Evaluate $\Delta AG(t) = \text{dist}(A(t), G)$, where $A(t)$ is incoming activation and G is the admissible geometry fixed by the frozen schema.
3. **Apply the admissibility gate.** Return block, constrain, admit, rest, or re-project according to the declared thresholds and schema rules.
4. **Act on the verdict.** Block contracts toward the Neurobaseline while preserving residuals; constrain reroutes or contracts through controller gains; admit projects activation into G and updates the state through $\Delta v(t) = (\Pi_G(A(t)) - x(t)) / \Delta t$.
5. **Check circular re-expression.** If the updated state requires boundary-preserving circular expression, evaluate the circular component Δv_π against the bounded π -band rather than treating π as identical to Δv .
6. **Log and update the brim.** Record the claim state, residuals, verdict, controller action, and brim-state update so the Holistic Governor can evaluate deformation, cost, drift, and policy tension in the next pass.

C.8.2 Operator Summary

The runtime cycle can be summarized by the paper's core operator sequence:

$$\Delta AG(t) = \text{dist}(A(t), G) \rightarrow \Delta v(t) = (\Pi_G(A(t)) - x(t)) / \Delta t$$

$$\rho(t) = A(t) - \Pi_G(A(t)); |\Delta v_\pi - \pi| \leq \varepsilon_\pi$$

In this summary, ΔAG names the distance-to-admissibility test, Π_G performs geometric projection, $\rho(t)$ preserves mismatch as evidence, Δv names projected admissible motion, and Δv_π denotes the circular re-expression component evaluated against the π -band. The notation therefore preserves the paper's key separation: the gate decides admissibility, the projection defines the next valid state, the residual keeps strain visible, and the π condition remains a bounded future controller hypothesis.

C.8.3 Implementation Notes for Future Simulation

Future simulation should specify the pieces that this appendix leaves intentionally open:

- o **State dimensionality:** define which envelope signals are included in $x(t)$ and how they are sampled.
- o **Projection:** replace scalar teaching examples with a projection onto a higher-dimensional admissible region G .
- o **Residual preservation:** store residuals in a versioned structure so frozen-schema comparisons and falsification tests can inspect what the gate did not resolve.
- o **Controller gains:** test contraction, rerouting, escalation, residual, and π -related gains under fixed evaluation conditions.
- o **π -band hypothesis:** treat circular re-expression as a bounded hypothesis to be tested for stability, coherence, and cost behavior, not as a proven law.
- o **Claim-state ledger:** log admissibility verdicts, residuals, controller actions, brim updates, and failure modes for ablation and governance review.

- o **Holistic Governor integration:** connect brim-state updates to drift detection, overload sensing, and policy-tension evaluation.

C.9 Failure Modes and Validation Conditions

This final section turns the closed-loop scaffold into a falsification target. C.8 defines how the controller should cycle; C.9 defines how that cycle can fail, how the system should respond, and what future simulations must test. The purpose is not to claim that the controller is stable today, but to make instability visible enough to be measured.

C.9.1 Failure Modes and Recovery Paths

The controller should be evaluated against explicit failure modes. Each mode below identifies a condition that would weaken the closed-loop claim and a bounded recovery path that preserves residual evidence rather than hiding instability.

1. Geometric Mismatch

A large ΔAG value indicates that activation has moved outside the admissible geometry G . The system should not automatically treat every mismatch as catastrophic; it should apply the verdict specified by the frozen schema.

- o If the mismatch is beyond the blocking threshold, contract toward the Neurobaseline and preserve the residual.
- o If the mismatch is near the boundary, constrain, project, or reroute according to the declared admissibility band.
- o Record the verdict, residual, and recovery action in the claim-state ledger.

2. Persistent Constraint Violations

Repeated constrain verdicts indicate that activation is approaching the brim in a direction that remains locally useful but globally misaligned.

- o Reroute through adjacent coherent V-Channels before allowing further outward stretch.
- o Reduce expansion pressure through contraction, concision, or lower-cost routing gains.
- o Escalate to the Holistic Governor if repeated constraints become systemic.

3. Projection Instability

Projection instability occurs when Π_G produces oscillation, divergence, or repeated repair loops instead of a stable admissible state.

- o Reduce controller gains and apply bounded contraction before the next projection attempt.
- o Log the instability as a failure condition for later schema review rather than changing the frozen schema during the run.
- o Re-evaluate the admissible manifold in a future revision cycle if the failure repeats across trials.

4. Circular Re-Expression Failure

Circular re-expression fails when Δv_π repeatedly falls outside the declared π -band after the state has already passed the admissibility gate.

- o Apply bounded contraction before attempting re-expression again.

- o Reduce the circular component of projected motion rather than treating π as identical to Δv .
- o Record the failure as evidence for future π -band calibration.

5. Residual Accumulation

Residual accumulation indicates that repeated projections or constraints are leaving unresolved strain between activation and admissible geometry.

- o Trigger claim-state review and residual comparison across runs.
- o Distinguish harmless residuals from accumulating deformation that affects quality, cost, policy friction, or coherence.
- o Use the preserved residual record to decide whether later schema revision, ablation, or controller redesign is warranted.

6. Supervisory Drift

Supervisory drift occurs when brim-state updates show long-term overload, channel instability, policy tension, or degraded quality despite locally valid actions.

- o Escalate to the Holistic Governor for whole-field interpretation.
- o Apply global contraction, reduce reasoning radius, or rebalance String activation.
- o Require the next simulation pass to report whether drift was reduced, displaced, or hidden.

These failure modes keep the controller falsifiable. A valid recovery path must preserve evidence, respect the frozen schema during the run, and make unresolved instability available for later review.

C.9.2 Simulation Scenarios and Test Conditions

Future validation should exercise the controller under controlled scenarios that test admissibility logic, projection stability, residual preservation, circular re-expression, and schema discipline.

Scenario	Initial condition	Expected behavior	Validation focus
Admissible activation	$x(0)$ lies near Π_G .	ΔAG remains small, the gate admits activation, and Δv produces smooth admissible motion.	Baseline admission and stable projection.
Boundary violation	$x(0)$ lies outside admissible geometry.	The gate returns constrain or block according to the frozen schema, residuals are preserved, and contraction or rerouting occurs.	Threshold discipline and residual preservation.
Repeated constraint	Activation repeatedly approaches the brim along a misaligned direction.	The controller reroutes, contracts, or escalates rather than allowing unbounded outward stretch.	Recovery from persistent near-boundary strain.
Projection instability	Π_G produces oscillatory or divergent updates.	Controller gains are reduced, contraction is applied, and the instability is logged for later schema or manifold review.	Stability under projection and repair loops.

Circular re-expression trigger	Δv produces a circular component near the brim.	Δv_π is evaluated against the π -band, and failed alignment triggers bounded contraction or later calibration evidence.	Bounded π hypothesis without asserting $\pi = \Delta v$.
Schema falsification test	Activation repeatedly fails under a frozen schema.	The claim-state ledger accumulates evidence, residuals remain inspectable, and future revision or ablation is justified only after the run.	Falsification discipline and prevention of post-hoc threshold movement.

Together, these scenarios form a compact validation matrix for future simulation. A successful controller should not merely pass favorable cases; it should expose its own failure modes, preserve residual evidence, and demonstrate that recovery reduces deformation rather than concealing it.

Taken as a whole, the runtime loop and validation scaffold make the closed-loop claim explicit: the system is closed only when every outward motion returns as evidence to the brim, every governance decision updates the next state, and every unresolved residual remains available for review. C.9 completes this scaffold by specifying the conditions under which the controller should fail visibly rather than succeed rhetorically. The loop becomes an engineered object only after its functions are specified, tested, and shown to remain stable under load. Until then, Appendix C should be read as a proposed bridge from bounded closed-loop governance to a formal controller implementation.

Acknowledgements

The Author gratefully acknowledges Microsoft for developing tools and platforms that support independent research, structured thinking, and the refinement of complex technical ideas. This Version 2.4.0 technical manuscript is offered as a conceptual research contribution for further critique and exploration around adaptive AI behavior, compute-aware governance, and responsible system design. The Author also acknowledges the use of Microsoft Copilot as an editorial aid during revision, including support for drafting flow, structural review, language refinement, and terminology consistency. All scientific concepts, architectural models, mathematical interpretations, system terminology, diagrams, and original theoretical contributions were created, selected, and directed by the Author, who remains solely responsible for the paper's claims, framing, and conclusions.
